

Firefly AI Pet 插件生态扩展介绍

本文档为 Firefly AI Pet 项目（AI Agent 赛道参赛作品）的正式附件，介绍其插件生态扩展的架构设计与发布现状。全文内容依据插件扩展包内的源码与接口契约文档整理，仅描述已实现并经测试验证的能力。

1 插件生态概览：从 AI 桌宠到可扩展 Agent 平台

Firefly AI Pet 以桌宠形态作为交互入口，内核是一套完整的 Agent 运行时，负责多轮对话、长期记忆管理与技能调度。在此基础上，项目通过插件扩展层（Plugin Extension Layer）将外部能力以统一契约接入系统，使产品从单一 AI 桌宠演进为可扩展的 Agent 平台。

插件扩展层承担四项职责：定义统一插件契约（FireflyExtension v2），规范插件从初始化、启动到卸载的完整生命周期；在宿主启动时自动发现并加载插件根目录下的合规插件；为每个插件提供能力声明（QuickToolManifest）与状态发布通道，使插件能力统一呈现在 Quick Tools 面板中；实施能力门控与启用管理，保证插件行为始终处于宿主与用户的控制之下。

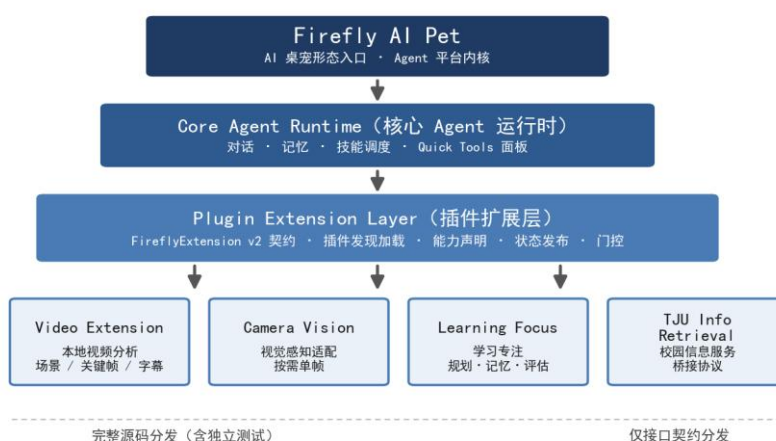


图 1 插件扩展层四层架构

如图 1 所示，四个首批插件覆盖多媒体处理、视觉感知、学习辅助与校园信息服务四个方向；其中三个插件以完整源码分发，一个插件以接口契约文档分发。

2 插件能力总览

表 1 四大插件功能定位与公开范围

插件名称	功能定位	公开范围	状态
firefly_video_extension	本地视频分析：时长统计、场景检测、关键帧提取、字幕识别 (复用宿主视频管线：FFmpeg、场景检测、OCR)	完整源码	可直接使用
firefly_camera_vision	摄像头视觉能力适配器：按需单帧采集、永不后台开启摄像头、设备状态声明	完整源码	可直接使用
learning_focus	学习专注：知识图谱规划、学习者画像记忆、作答证据评估(算法层纯标准库，附独立测试集)	完整源码 + 测试	可直接使用
tju_info_retrieval	天津大学信息检索薄适配器：以子进程桥接协议连接外部私有检索工程	仅接口契约文档	文档分发

四个插件遵循同一契约，由同一套发现与门控机制管理，差异仅在于能力方向与分发方式：前三个插件公开全部实现，可直接部署使用；tju_info_retrieval 的核心检索工程为私有项目，本包公开其宿主侧完整接入契约，供按契约复现同构桥接。

3 插件架构设计

插件发现与 Plugin Root。 宿主启动时扫描插件根目录（默认位于用户数据目录的 plugins 子目录，可通过环境变量或路径配置文件重定向），将同时满足两个条件的目录识别为插件：目录为 Python 包（含 __init__.py），且其 plugin.py 暴露 create_plugin(parent=None) 工厂函数。

统一契约。 插件实现 FireflyExtension v2 契约，遵循 initialize、start、open、stop、shutdown 的生命周期；能力与元数据通过内嵌 manifest 声明，无需额外配置文件。Quick Tools 面板据此渲染插件入口与实时状态。

名单管理与 Enable 开关。 宿主维护插件禁用名单，命中名单的插件在加载阶段即被拒绝；每个插件的启用状态独立持久化，用户可逐插件开关；插件被调用前还需通过能力门控检查，确保仅在相应能力可用时暴露入口。

Fail-closed 设计。 加载链上任何环节不通过——包结构不符、工厂缺失、返回类型不符、命中禁用名单——该插件即被拒绝加载，异常被隔离在插件边界内，宿主运行时不受影响。

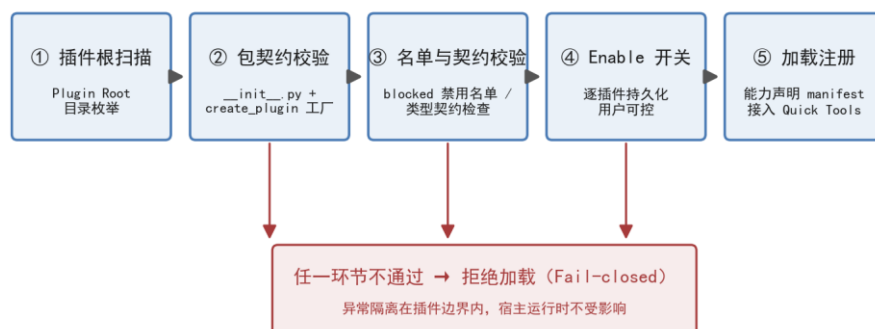


图 2 插件加载的 fail-closed 门控链

需要强调的是：插件不是简单脚本，而是 Agent 能力扩展接口。插件以契约化方式接入运行时的生命周期、状态总线与 UI 入口，其行为可被声明、被查询、被开关、被审计，这是平台化扩展与脚本堆叠的本质区别。

4 典型插件介绍

4.1 Learning Focus：学习专注插件

Learning Focus 由三个自研算法模块构成。规划层实现最小知识图谱（节点分层与拓扑排序）与加权任务规划，依据学习者状态在诊断、讲解、测验、复习之间调度；记忆层维护学习者画像（原子写持久化）与追加式事件日志，并定义可替换的记忆后端协议，可无缝切换至宿主记忆系统；评估层将作答证据折算为掌握度档位，结合遗忘曲线安排复习节奏。其算法层为纯标准库实现，附带独立测试集，可在脱离宿主的环境下单独运行验证。

4.2 Vision / Video：感知与多媒体扩展

视频扩展插件复用宿主视频管线，对本地视频完成时长统计、场景检测、关键帧提取与字幕识别；当前版本明确不启用视觉模型，不写入记忆、不新增事件类型，

能力边界克制。摄像头视觉插件是零资源持有的能力适配器：自身不拥有摄像头、工作线程与持久化，仅在设备可用性层面声明状态，生产采集路由保留在宿主原有通路中。两者共同体现感知类扩展的设计原则：显式请求、按需触发、即用即弃，摄像头永不后台开启。

4.3 TJU Info Retrieval：桥接协议与接口化设计

该插件面向校园学术信息服务场景，采用薄适配器架构：适配器自身零检索逻辑，不重建索引、不做排序、不持有存储。每个请求通过一次性隔离子进程调用外部工程的桥接 CLI（健康检查、登录态探测、手动登录、检索四类命令），以 JSON 协议返回并归一化为统一结果模型；插件维护 READY、AUTH_REQUIRED、OFFLINE、ERROR 四态状态机，状态查询为零副作用的本地检查。其核心检索工程为私有项目，本包仅公开接口契约文档、不公开内部适配实现，体现”能力可集成、实现可私有”的接口化设计。

5 安全与工程化

插件扩展包在发布前经过系统脱敏检查，满足以下安全边界：

- 无 API Key、无 Token：包内不含任何密钥、令牌或连接串；
- 无 Cookie、无登录凭据：TJU 登录态归属外部工程，适配器不读取、不存储、不传递任何凭据，登录仅走受控浏览器的手动流程；
- 无用户数据、无个人隐私信息：包内不含用户画像、会话记录等任何运行期数据；
- 无本机绝对路径：安装位置由环境变量或配置文件决定，包内不硬编码任何机器相关路径。

在工程化层面：摄像头插件从设计上保证永不后台开启；子进程桥接在启动前剥离宿主图形环境变量，避免污染外部图形程序；桥接诊断日志有界轮转；learning_focus 数据写入采用原子写与追加式日志，支持重放恢复。插件包经过脱敏检查，可用于公开展示；本文档亦经过相同标准的安全扫描后发布。